

POLYGLOT MEET

Real-Time Multilingual Video Conferencing

Architecture & Build Plan

Version 1.0 • July 15, 2026

Backend: Go + Pion WebRTC • AI: Google Gemini • Frontend: Vanilla JS

Infra: Docker Compose + coturn

1. Executive Summary

Polyglot Meet is a browser-based real-time video conferencing application that eliminates the language barrier from live conversation. Each participant selects the language they want to hear. Speech from every other participant is transcribed, translated, and rendered back — as live captions (Phase 1) or a dubbed voice track (Phase 2) — with under 3 seconds end-to-end latency.

Aspect	Detail
Problem	Multilingual teams can't meet naturally without human interpreters.
Solution	Per-listener real-time translated captions + optional dubbed audio.
Differentiator	Each listener picks their own language independently — not one global setting.
V1 Scope	1:1 and small group calls (<6), live subtitles, 2–4 language pairs.
Tech Stack	Go + Pion, Vanilla JS, Gemini APIs (STT / Translate / TTS), Docker.
Latency Target	< 3s for captions, < 5s for dubbed audio (end-to-end).

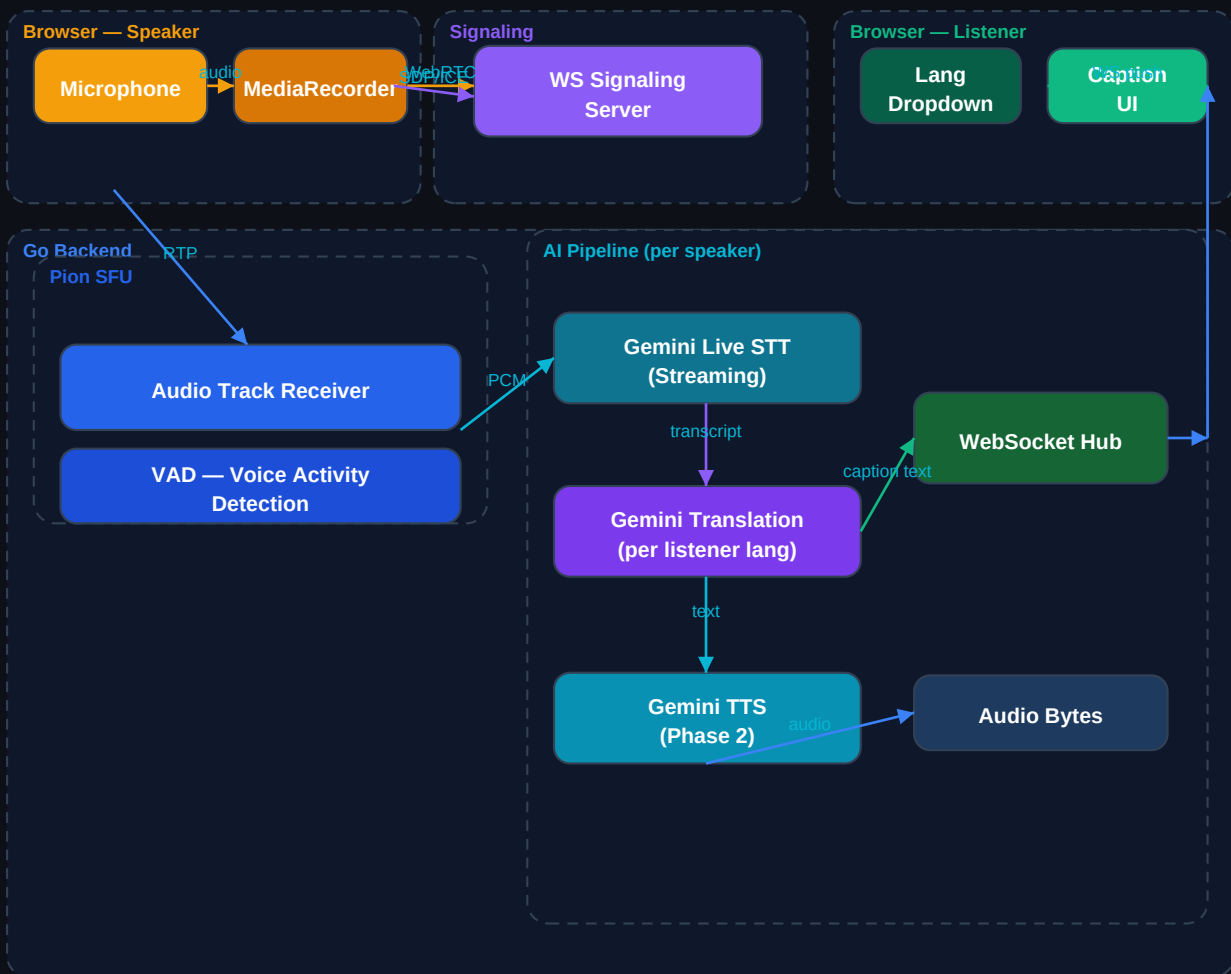
2. Technology Stack

Layer	Technology	Purpose
Backend	Go (Golang)	Core server, room management, pipeline orchestration
WebRTC	Pion (Go)	SFU — receive and route audio tracks server-side
Signaling	WebSocket (gorilla/websocket)	SDP/ICE exchange, language preference, caption push
AI — STT	Gemini Live API (gemini-2.0-flash-live)	Streaming audio → partial + final transcripts
AI — Translate	Gemini Text API (gemini-2.0-flash)	Translate final transcript per listener's language
AI — TTS	Gemini TTS (gemini-2.5-flash-preview-tts)	Texts → audio bytes per listener (Phase 2)
AI — Voice Clone	Gemini Multi-Speaker TTS	Cloned voice dubbing (Phase 3)
Frontend	Vanilla JS + HTML/CSS	No framework; native RTCPeerConnection + MediaRecorder
TURN	coturn (Docker)	NAT traversal for WebRTC peers
STUN	Google public STUN	ICE candidate discovery
Containerization	Docker + Docker Compose	Local dev and deployment packaging

3. Project Structure

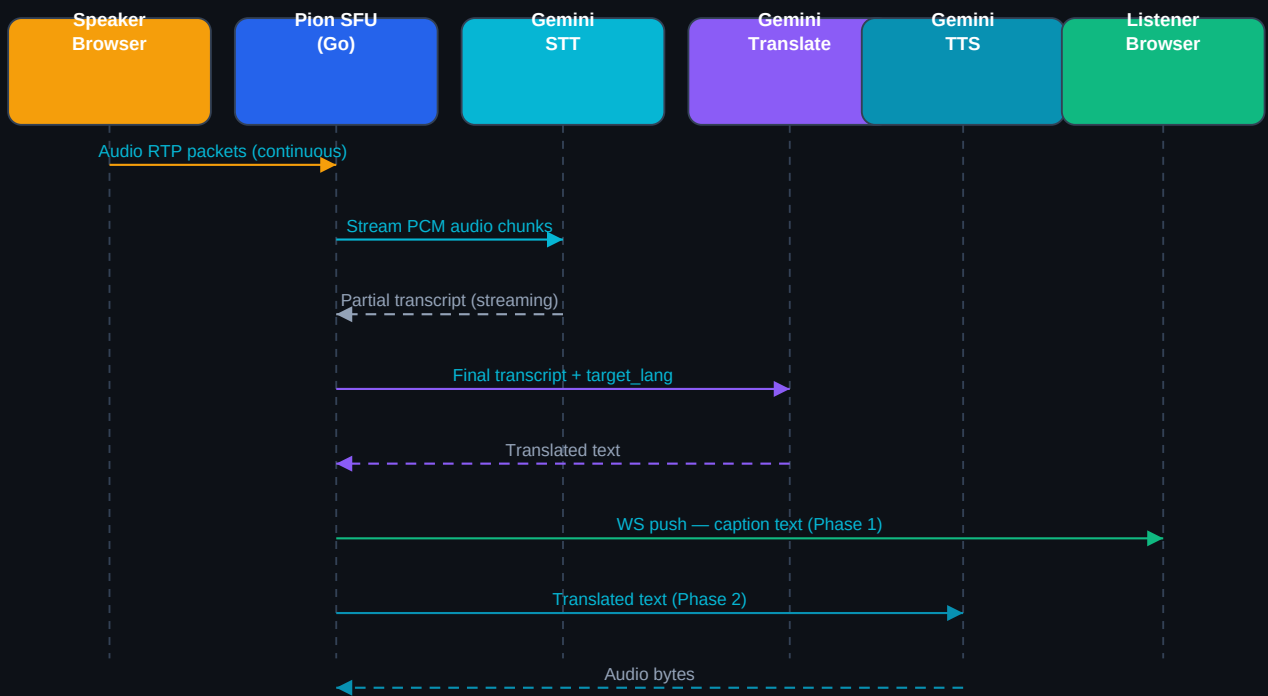
polyglot-meet/		
■■■ docker-compose.yml		
■■■ backend/		
■	■■■ main.go	# Entry point
■	■■■ signaling/	# WebSocket signaling server
■	■■■ sfu/	# Pion SFU – audio track routing
■	■■■ pipeline/	# STT → translate → TTS orchestration
■	■■■ ai/	
■	■ ■■■ interface.go	# AIProvider interface
■	■ ■■■ gemini.go	# Gemini implementation
■	■■■ room/	# Room + participant state (in-memory)
■	■■■ config/	# Env, API keys
■■■ frontend/		
■	■■■ index.html	# Join / lobby page
■	■■■ call.html	# Active call UI
■	■■■ js/	
■	■ ■■■ webrtc.js	# RTCPeerConnection, MediaRecorder
■	■ ■■■ signaling.js	# WebSocket client
■	■ ■■■ captions.js	# Caption bubble renderer
■	■ ■■■ audio.js	# Dubbed audio playback + ducking
■	■■■ css/style.css	
■■■ turn/		
	■■■ coturn.conf	

4. Full System Flow Diagram



The speaker's audio is captured via MediaRecorder and transmitted through WebRTC to the Pion SFU. The SFU extracts audio packets, runs VAD to detect active speech, and streams PCM chunks to Gemini Live STT. Transcripts are translated per-listener via Gemini Translate. Captions are pushed over WebSocket (Phase 1). In Phase 2, translated text is synthesized by Gemini TTS and audio bytes are pushed to each listener's browser.

5. AI Pipeline — Sequence Diagram (per utterance)



Step	Latency Budget	Service
Audio capture → Pion	~50ms	WebRTC / RTP
Pion → Gemini STT (streaming)	~300ms	Gemini Live API
STT → Translation	~200ms	Gemini Text API
Translation → Caption WS push	~20ms	WebSocket
Translation → TTS (Phase 2)	~600ms	Gemini TTS API
TTS audio → Listener WS	~50ms	WebSocket
TOTAL (captions)	< 600ms	Target: < 3s end-to-end
TOTAL (dubbed audio)	< 1.2s	Target: < 5s end-to-end

6. Gemini API Usage Map

Task	Gemini Model	Mode	Phase
STT	gemini-2.0-flash-live	Streaming audio in → text out	1, 2, 3
Translation	gemini-2.0-flash	Single prompt per utterance	1, 2, 3
TTS	gemini-2.5-flash-preview-tts	Text in → audio bytes out	2, 3
Voice Clone	Gemini Multi-Speaker TTS	Voice profile + text → cloned audio	3

Go AI Interface

All Gemini calls are abstracted behind a single Go interface. Swap provider by changing one line.

```
// ai/interface.go
type AIProvider interface {
    StreamSTT(ctx context.Context, audioStream io.Reader, lang string) (<-chan Transcript, error)
    Translate(ctx context.Context, text, sourceLang, targetLang string) (string, error)
    Synthesize(ctx context.Context, text, lang string) ([]byte, error)
}
```

7. Phase Delivery Plan

Phase 1 — Subtitles MVP

- Pion SFU receives audio tracks from all participants
- Stream audio to Gemini Live STT → partial + final transcripts
- For each listener: Gemini Translate → target-language text
- Push caption text over WebSocket to correct listener only
- Frontend renders attributed caption bubbles per speaker
- Show 'translating...' indicator during processing
- EXIT CRITERIA: 2 users, different languages, captions < 3s delay

Phase 2 — Dubbed Audio (Generic TTS Voice)

- After translation, call Gemini TTS → get audio bytes
- Push audio bytes over WebSocket to listener's browser
- Frontend plays dubbed audio via Web Audio API

- Duck original speaker's RTP audio while TTS plays (GainNode)
- Echo prevention: TTS output routed to speakers only, not re-captured by mic
- Toggle UI: captions-only vs dubbed-audio mode
- EXIT CRITERIA: dubbed audio plays naturally, no feedback loops

Phase 3 — Voice Cloning & Polish

- Capture speaker voice sample during first 30s of call
- Register voice profile with Gemini Multi-Speaker TTS
- Use cloned voice for TTS synthesis instead of generic stock voice
- Opt-in consent UI per participant (required before cloning)
- Graceful fallback to generic TTS if cloning fails or is declined
- Visual context translation: use video frames for disambiguation (if supported)
- EXIT CRITERIA: cloned voice recognizably similar to original speaker

8. Docker Compose Services

Service	Image	Ports	Purpose
backend	golang:1.23-alpine (built)	8080	Go app — HTTP + WebSocket + Pion SFU
coturn	coturn/coturn:latest	3478, 5349	TURN server for WebRTC NAT traversal

No database needed for prototype — room state held in-memory in Go.

9. Non-Functional Requirements

Category	Requirement
Latency	< 3s captions, < 5s dubbed audio, measured end-of-utterance to delivery
Scalability	Prototype: ≤6 participants comfortably. Horizontal scaling deferred.
Reliability	Graceful degrade to captions-only if TTS/translate service is slow or down.
Security	Audio/text sent to Gemini not stored longer than necessary. Voice clone = explicit consent.
Compatibility	Chrome, Edge, Firefox, Safari — modern evergreen browsers with WebRTC support.
Observability	Log per-utterance: STT latency, translate latency, TTS latency, total latency, token cost.
Cost Tracking	Instrument Gemini API cost per call-minute from day one for pricing model.

10. Build Order

#	Task	Category
1	Go project scaffold + Docker Compose + coturn config	Foundation
2	Pion SFU + WebSocket signaling (SDP/ICE/room state)	WebRTC core
3	Vanilla JS frontend — join page + call UI + RTCPeerConnection	Frontend
4	Gemini STT integration — stream audio, receive transcripts	Phase 1 AI
5	Gemini Translation — translate per listener language	Phase 1 AI
6	Caption delivery via WebSocket + frontend bubble rendering	Phase 1 UX
7	Gemini TTS + audio bytes delivery + Web Audio API playback	Phase 2
8	Audio ducking + echo prevention + captions/audio toggle UI	Phase 2 UX

9	Voice cloning + consent flow + graceful fallback	Phase 3
---	--	---------

11. Risks & Mitigations

Risk	Impact	Mitigation
High latency makes conversation feel unnatural	High	Start with streaming STT + subtitles-only MVP; optimize before TTS
Gemini API costs scale fast with multi-language calls	High	Model cost per participant-language early; add rate limiting
Echo: translated audio re-captured by mic	Med	Route TTS audio to output only; strong echo cancellation
Overlapping speech breaks STT accuracy	Med	V1: one active speaker at a time; add diarization in V2
Gemini API rate limits / outages	Med	Abstract AI calls; add retry + fallback to captions-only

12. Success Metrics

Metric	Target
End-to-end caption latency	< 3 seconds
End-to-end dubbed audio latency	< 5 seconds
Call completion rate	> 80% of started calls complete without crash
Translation intelligibility (rated)	> 7/10 average in post-call user survey
Pilot users onboarded	10–20 individuals / 3–5 teams
Cost per call-minute tracked	Instrumented from day 1, ceiling set after real data

End of Document